MONASH University

Searching for Checkmate

Week 2 — Breadth-First and Depth-First Search

Julian Garcia

FIT1061 — Introduction to Artificial Intelligence

COMMONWEALTH OF AUSTRALIA | Copyright Regulations 1969 | WARNING: This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.



© Peter Morgan / Reuters

MILESTONE

Deep Blue vs Kasparov

May 11, 1997

The machine that beat the world chess champion.

What happened?



Feng-hsiung Hsu moves on behalf of Deep Blue (Reuters)

Watch: Kasparov resigns, Game 6

IBM's **Deep Blue** defeated Garry Kasparov—the greatest chess player alive—in a six-game match.

What happened?



Feng-hsiung Hsu moves on behalf of Deep Blue (Reuters)

Watch: Kasparov resigns, Game 6

IBM's **Deep Blue** defeated Garry Kasparov—the greatest chess player alive—in a six-game match.

Deep Blue evaluated **200 million positions per second**.

The question

How does a machine *search* through

millions of possibilities

and find the right move?

Before we search: the prep guide

The prep guide *What Is a Graph?* covered:

- Nodes and edges
- Adjacency lookups: which neighbours does this node have?
- Walls are nodes you skip

Today we'll search a **grid** — a graph where each cell connects to its neighbours (up, down, left, right).

We'll use a `Queue` (FIFO) and a `Stack` (LIFO) — both are standard building blocks.

Today

Games to graphs

Breadth-first search

BFS in action

Depth-first search

Beyond brute force

Games to graphs

Breadth-first search

BFS in action

Depth-first search

Beyond brute force

Games to graphs

A simpler game: tic-tac-toe

Depth 0

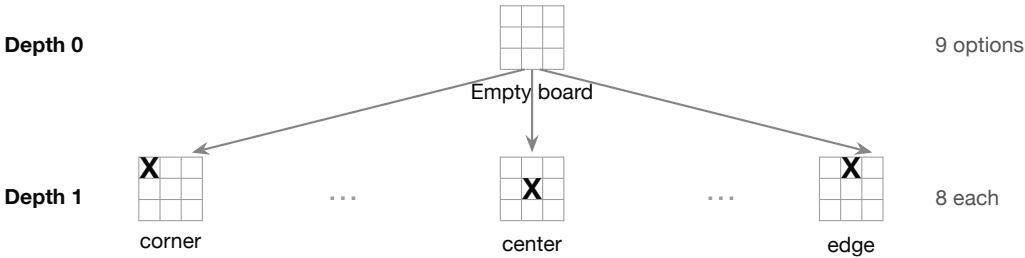


Empty board

9 options

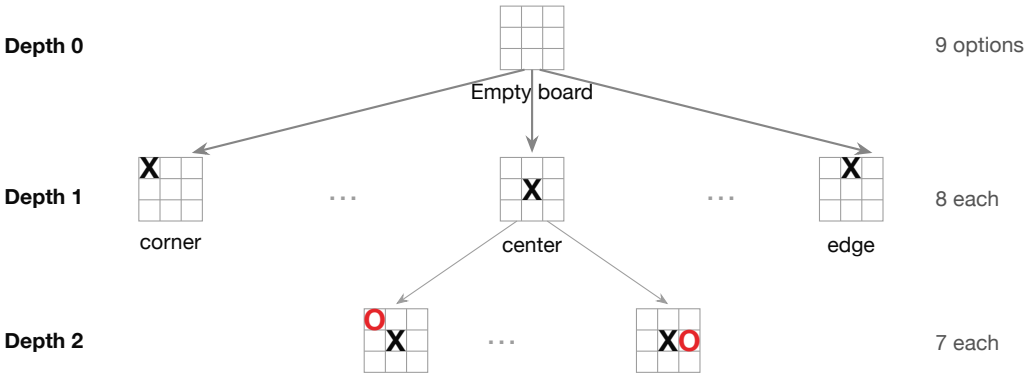
Start of every game.

A simpler game: tic-tac-toe

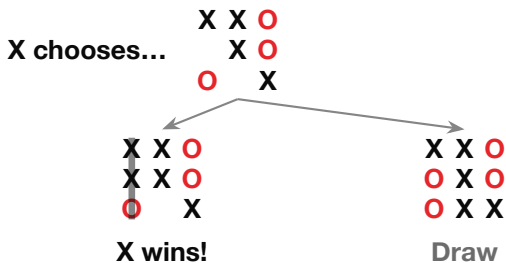


After X's first move.

A simpler game: tic-tac-toe



How a game ends: one branch



A **leaf**: someone wins or no moves remain. Search = find the path to a **winning leaf**.

The tic-tac-toe explosion

Depth	Positions	That's like…
1	9	a small group
2	$9 \times 8 = 72$	a classroom
4	$9 \times 8 \times 7 \times 6 = 3{,}024$	a lecture theatre
9	$9! = 362{,}880$	**a small suburb**

The tic-tac-toe explosion

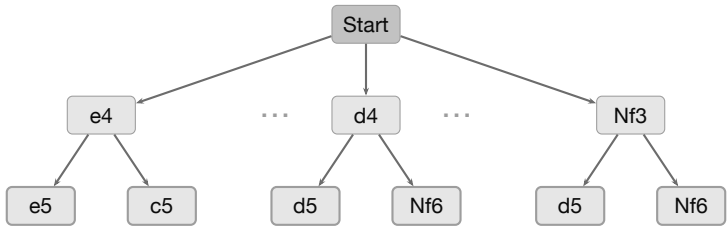
Depth	Positions	That's like…
1	9	a small group
2	$9 \times 8 = 72$	a classroom
4	$9 \times 8 \times 7 \times 6 = 3{,}024$	a lecture theatre
9	$9! = 362{,}880$	**a small suburb**

Time on a modern laptop

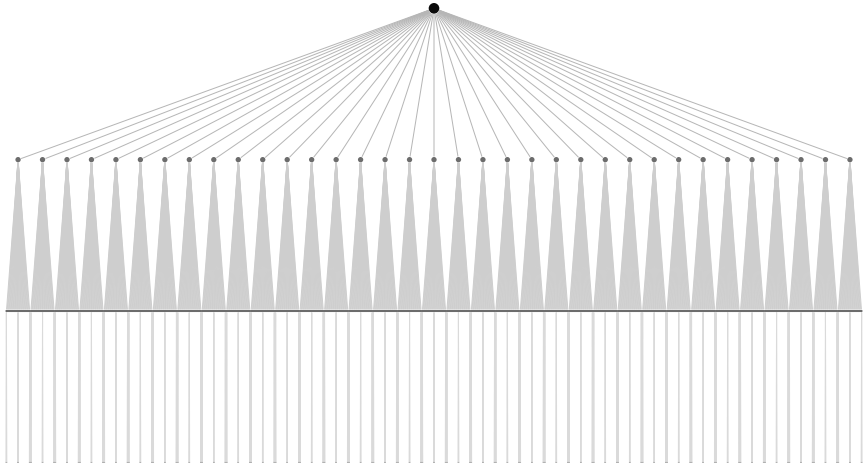
All 362,880 leaves searched in $\sim$0.04 s (at 10^7 ops/sec). Tic-tac-toe is **solved**.

Chess as a tree

Same idea, but the branching factor is ∼**35** instead of 9.



Only showing 3 of ∼35 branches. Real tree has ∼35 at *every* node.



∼42,875 positions at depth 3.

Problem formulation

Every search problem has four parts:

- **States** (s) — configurations of the world (board positions)

Problem formulation

Every search problem has four parts:

- **States** (s) — configurations of the world (board positions)
- **Actions** (a) — moves from one state to another (legal moves)

Problem formulation

Every search problem has four parts:

- **States** (s) — configurations of the world (board positions)
- **Actions** (a) — moves from one state to another (legal moves)
- **Successor function** — which state does an action take us to?

Problem formulation

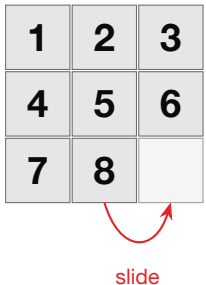
Every search problem has four parts:

- **States** (s) — configurations of the world (board positions)
- **Actions** (a) — moves from one state to another (legal moves)
- **Successor function** — which state does an action take us to?
- **Goal test** — are we done? (checkmate? reached destination?)

You'll use this formulation in every search task this semester. Edge weights and path cost $g(n)$ arrive later.

Russell & Norvig, AIMA 4e, §3.1 (Problem-Solving Agents).

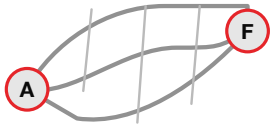
Example: 8-puzzle as search



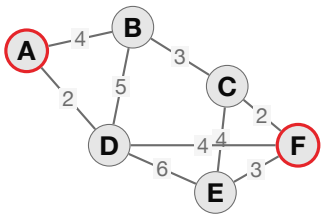
- **States:** 3×3 tile configurations
- **Actions:** slide blank up / down / left / right
- **Goal test:** tiles in sorted order?
- **Action cost:** 1 per slide

Search isn't about chess. It's about the four parts.

Example: GPS routing as search



abstracts to ↓



- **States:** intersections (A–F)
- **Actions:** drive along an edge
- **Goal test:** at F?
- **Action cost:** edge travel time (min)

Real maps become graphs.
Same four parts. Same algorithm.

Games to graphs

Breadth-first search

BFS in action

Depth-first search

Beyond brute force

Breadth-first search

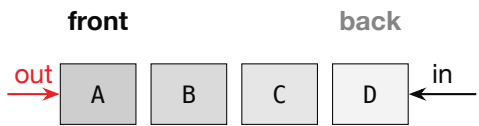
explore level by level

What if we explored **every option at this depth** (distance from start)

before going any deeper?

The queue: first in, first out

A **queue** processes items in the order they arrive.



BFS uses a queue as its **frontier**—the list of nodes waiting to be explored.

The BFS idea

Breadth-First Search

Explore all nodes at depth d before any node at depth $d + 1$.

The queue ensures this: nodes discovered first are explored first.

What BFS guarantees:

- Finds a path if one exists
- Path is the **shortest** (fewest steps)

The BFS idea

Breadth-First Search

Explore all nodes at depth d before any node at depth $d + 1$.

The queue ensures this: nodes discovered first are explored first.

What BFS guarantees:

- Finds a path if one exists
- Path is the **shortest** (fewest steps)

What BFS costs:

- Explores *many* nodes
- Memory grows with width

Breadth-First Search

Breadth-First Search

```
function BFS(start, goal):
    frontier ← Queue()
    frontier.enqueue(start)
    while frontier is not empty:
        current ← frontier.dequeue()
        if current = goal:
            return success
        for each neighbour of current:
            frontier.enqueue(neighbour)
    return failure
```

Notation: `x <- y` means *"x gets the value y"*. The only data structure here is a `Queue`.

Poll

We run this search on a graph that has a loop in it.

What happens?

- **It finds the goal and stops**
- **It finds the goal but wastes effort**
- **It never stops**
- **It misses the goal**

Vote now — scan the code for your campus, *or* type the address.



Melbourne

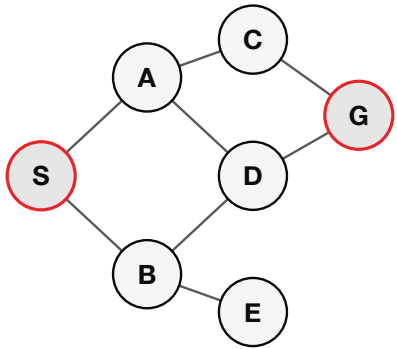
pollev.com/juliangarcia



Malaysia

pollev.com/onghueyfang767

Trace it: BFS from S to G



Track the queue contents at each step.

What went wrong?

We re-enqueued.

Nothing stopped us from adding S, A, B, D back into the frontier after they had already been processed. The queue keeps growing.

On a graph with cycles **and no escape**, the queue **never empties** — the search **never terminates**.

Key Takeaway

We need a way to remember what we have already seen, so we never explore the same node twice.

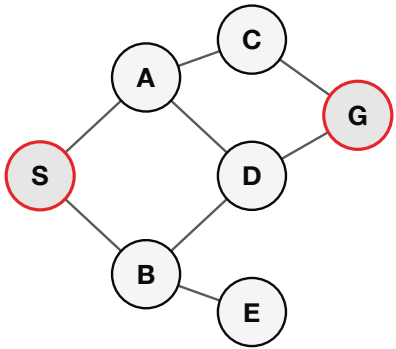
BFS, refined

Breadth-First Search

```
function BFS(start, goal):
    frontier ← Queue()
    frontier.enqueue(start)
    visited ← {start}
    while frontier is not empty:
        current ← frontier.dequeue()
        if current = goal:
            return success
        for each neighbour of current:
            if neighbour not in visited:
                visited.add(neighbour)
                frontier.enqueue(neighbour)
    return failure
```

One new structure: `visited` (a set — fast membership lookup, no duplicates). Initialise with `start`; check membership before enqueueing.

Trace it again: BFS from S to G



What's still broken?

Better, but still incomplete

Key Takeaway

BFS with `visited` **terminates** and finds G. No more re-enqueueing; no more infinite loops.

Better, but still incomplete

Key Takeaway

BFS with `visited` **terminates** and finds G. No more re-enqueueing; no more infinite loops.

But: we know only *that* G is reachable. We threw away every step that led there.

We want the **path** — the sequence of moves from S to G — not a boolean. We need to record, as we go, which node led to which.

Better, but still incomplete

Key Takeaway

BFS with `visited` **terminates** and finds G. No more re-enqueueing; no more infinite loops.

But: we know only *that* G is reachable. We threw away every step that led there.

We want the **path** — the sequence of moves from S to G — not a boolean. We need to record, as we go, which node led to which.

One more structure: a dict that maps each discovered node to its predecessor.

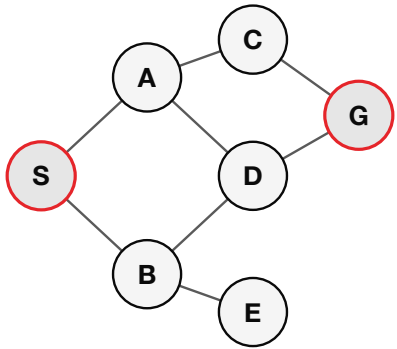
BFS, final

Breadth-First Search

```
function BFS(start, goal):
    frontier ← Queue()
    frontier.enqueue(start)
    visited ← {start}
    came_from ← {start: nil}
    while frontier is not empty:
        current ← frontier.dequeue()
        if current = goal:
            return reconstruct_path(came_from, start, goal)
        for each neighbour of current:
            if neighbour not in visited:
                visited.add(neighbour)
                came_from[neighbour] ← current
                frontier.enqueue(neighbour)
    return failure
```

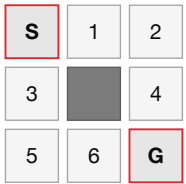
Path reconstruction walks the `came_from` chain backwards from `goal` to `start` and reverses the sequence.

Trace it: BFS with `came_from`

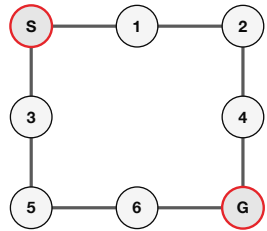


Reconstruct the path.

Grids *are* graphs



3×3 grid; one wall at the centre



Each walkable cell = one node (same number). Edges = step to a 4-neighbour. Undirected: BFS does not care about direction here.

Key Takeaway

BFS does not care whether the input “looks like” a grid, a maze, or a road network. It only needs one operation: *given a node, return its neighbours*.

Why does BFS find the shortest path?

Key Takeaway

BFS processes nodes in **order of distance** from the start, where every edge counts the same (*unweighted* graph).

All nodes at distance 1 are explored before any node at distance 2. All at distance 2 before any at distance 3. And so on.

So the *first* time we reach the goal, it must be via the shortest path.

Why does BFS find the shortest path?

Key Takeaway

BFS processes nodes in **order of distance** from the start, where every edge counts the same (*unweighted* graph).

All nodes at distance 1 are explored before any node at distance 2. All at distance 2 before any at distance 3. And so on.

So the *first* time we reach the goal, it must be via the shortest path.

This works because:

- The **queue** enforces FIFO order

Why does BFS find the shortest path?

Key Takeaway

BFS processes nodes in **order of distance** from the start, where every edge counts the same (*unweighted* graph).

All nodes at distance 1 are explored before any node at distance 2. All at distance 2 before any at distance 3. And so on.

So the *first* time we reach the goal, it must be via the shortest path.

This works because:

- The **queue** enforces FIFO order
- Every step costs the same (unweighted graph)

Games to graphs

Breadth-first search

BFS in action

Depth-first search

Beyond brute force

BFS in action

Small grid: 5×5



Path found	shortest (guaranteed)
Cells explored	nearly all open cells
Grid cells	25 (5 walls)

explored path wall

BFS explores **everything in its radius**.

What if we changed the **frontier**?

Games to graphs

Breadth-first search

BFS in action

Depth-first search

Beyond brute force

Depth-first search

deep before wide

Swap the queue for a stack

Depth-First Search

Same algorithm as BFS. Replace the queue with a **stack**.

Stack = LIFO: pop the *newest* node first.

BFS (queue, FIFO)

- explores layer by layer
- finds **shortest** path
- frontier holds whole layer

DFS (stack, LIFO)

- dives along one branch
- finds *a* path (not shortest)
- frontier holds one branch

One data-structure swap. The rest of the code is **identical**.

DFS — one data-structure swap

Depth-First Search

```
function DFS(start, goal):
    frontier ← Stack()
    frontier.push(start)
    visited ← {start}
    came_from ← {start: nil}
    while frontier is not empty:
        current ← frontier.pop()
        if current = goal:
            return reconstruct_path(came_from, start, goal)
        for each neighbour of current:
            if neighbour not in visited:
                visited.add(neighbour)
                came_from[neighbour] ← current
                frontier.push(neighbour)
    return failure
```

Three substitutions in BFS: `Queue` → `Stack`, `enqueue` → `push`, `dequeue` → `pop`. The `visited` and `came_from` logic is **identical**. Same algorithm, different exploration order.

Same maze, different paths

Run BFS and DFS from the same start on a maze with a long dead-end corridor.

	BFS	DFS
Path length	**shorter (optimal)**	**longer**
Exploration shape	radial wave	corridor / winding line

DFS commits to one branch, dead-ends, backtracks — it walks a **longer** path. BFS spreads outward and finds the goal on the **shortest** layer.

Games to graphs

Breadth-first search

BFS in action

Depth-first search

Beyond brute force

Beyond brute force

smarter, not bigger

Poll

To look 10 moves ahead in chess, a laptop would need to check about 3 quadrillion positions, at 10 million each second.

How long is that?

- **About an hour**
- **About a month**
- **About 9 years**
- **About 1,000 years**

Vote now — scan the code for your campus, *or* type the address.

Melbourne
pollev.com/juliangarcia

Malaysia
pollev.com/onghueyfang767

Can BFS or DFS play chess?

Both algorithms explore roughly b^d positions: branching factor b, depth d.

Chess: $b \approx 35$ legal moves per position.

Depth	Positions ($b = 35$)	That's like…
1 move	35	a classroom

Can BFS or DFS play chess?

Both algorithms explore roughly b^d positions: branching factor b, depth d.

Chess: $b \approx 35$ legal moves per position.

Depth	Positions ($b = 35$)	That's like…
1 move	35	a classroom
2 moves	$35^2 = 1{,}225$	a lecture hall

Can BFS or DFS play chess?

Both algorithms explore roughly b^d positions: branching factor b, depth d.

Chess: $b \approx 35$ legal moves per position.

Depth	Positions ($b = 35$)	That's like…
1 move	35	a classroom
2 moves	$35^2 = 1{,}225$	a lecture hall
4 moves	$35^4 \approx 1.5$M	population of Adelaide

Can BFS or DFS play chess?

Both algorithms explore roughly b^d positions: branching factor b, depth d.

Chess: $b \approx 35$ legal moves per position.

Depth	Positions ($b = 35$)	That's like…
1 move	35	a classroom
2 moves	$35^2 = 1{,}225$	a lecture hall
4 moves	$35^4 \approx 1.5$M	population of Adelaide
10 moves	$35^{10} \approx 2.7 \times 10^{15}$	**7× grains of sand on Bondi Beach**

Can BFS or DFS play chess?

Both algorithms explore roughly b^d positions: branching factor b, depth d.

Chess: $b \approx 35$ legal moves per position.

Depth	Positions ($b = 35$)	That's like…
1 move	35	a classroom
2 moves	$35^2 = 1{,}225$	a lecture hall
4 moves	$35^4 \approx 1.5$M	population of Adelaide
10 moves	$35^{10} \approx 2.7 \times 10^{15}$	**7× grains of sand on Bondi Beach**

Time at 10^7 nodes/sec (laptop): ∼9 years — for either algorithm.

Neither algorithm scales. We need a *smarter* strategy.

Next week: can we steer the search? **Greedy best-first** — give the algorithm a sense of direction.

Your phone vs Deep Blue

Deep Blue (1997)

- Refrigerator-sized supercomputer
- $\sim 2 \times 10^8$ positions/sec
- Elo $\sim$2,850
- Won one match. Dismantled.

Stockfish on your phone (2026)

- Fits in your pocket
- $\sim 10^7$ nodes/sec on a phone
- Elo $\sim$**3,500+**
- Beats Deep Blue *easily*

Your phone vs Deep Blue

Deep Blue (1997)

- Refrigerator-sized supercomputer
- $\sim 2 \times 10^8$ positions/sec
- Elo $\sim$2,850
- Won one match. Dismantled.

Stockfish on your phone (2026)

- Fits in your pocket
- $\sim 10^7$ nodes/sec on a phone
- Elo $\sim$**3,500+**
- Beats Deep Blue *easily*

Key Takeaway

Hardware shrank. But that's not why your phone is stronger.

Beyond uninformed search

- **Heuristics + pruning** arrive next. **Greedy best-first** gives the search a sense of direction; **minimax** prunes adversarial game trees by skipping branches that cannot change the outcome.
- Adversarial game-play uses depth-limited minimax with a board evaluator — a strong starting point if you want to push further.

The question Deep Blue raises

Discussion

After losing Game 6, Kasparov accused IBM of cheating—of having human grandmasters secretly guide the machine between games.

IBM refused a rematch. They dismantled Deep Blue.

We still don't know exactly what happened in those games.

Your reflective response (150–250 words):
When an AI wins, does it matter how?

This week’s debate

Motion

Deep Blue’s win tells us nothing about intelligence.

Take a side on Kialo (a debate app) — for or against.

“The computer doesn’t play chess like a human, any more than an airplane flies like a bird.”

— Feng-hsiung Hsu, Deep Blue architect

In the lab this week

Pass tier. **Aiming for a Pass**

1. Hand-trace BFS on a small graph (show the frontier at each step)
2. Implement BFS from the lecture pseudocode
3. Adapt to DFS by swapping the queue for a stack
4. Compare BFS vs DFS on a maze with a long dead-end corridor (path length, nodes expanded)
5. Reflect on Deep Blue

Credit tier. **Aiming for a Credit** — all of the above, plus: think about how different graph shapes change which algorithm wins.

Distinction tier. **Aiming for a Distinction** — all of the above, plus: think about what a cheap, never-pessimistic estimate of remaining cost could look like.

BFS spreads. DFS dives.

Next week: what if the search had a

sense of direction?